Лабораторная работа № 5

Аналитическая обработка данных в Oracle Database 11g Express Edition

1. Цель работы

Изучение средств аналитической обработки данных в Oracle Database 11g Express Edition.

2. Задачи

Создание представления для таблицы фактов хранилища данных. Создание и выполнение SQL-запросов для аналитической обработки данных с использованием агрегатных функций, функций перекрестного анализа таблиц, ранжирования, подсчета долей, статистических функций в БД Oracle Database 11g Express Edition.

3. Теоретическая часть

3.1. Общие сведения об аналитических функциях

СУБД Oracle (начиная с версии Oracle9i) предоставляет в распоряжение пользователей отличные высокопроизводительные функции, позволяющие выполнять сложный анализ данных в хранилищах данных. Вся эта функциональность сосредоточена в базе данных, является частью стандартного продукта Oracle и позволяет выполнять анализ данных с помощью **аналитических функций** таких типов:

- агрегатные и статистические функции,
- функции перекрестного анализа таблиц,
- функции упорядочения (ранжирования),
- функции подсчета долей,
- функции обработки методом окна,
- функции запаздывания и опережения,
- дополнительные статистические функции.

Существует несколько новых понятий, используемых аналитическими функциями.
Порядок обработки (Processing Order). Запросы, использующие аналитические функции, обрабатываются в три стадии. Первая включает все соединения таблиц, инструкции WHERE, GROUP BY и HAVING для агрегатных и статистических функций (групповых запросов). Вторая — применение аналитических функций к результирующему множеству. Третья — если есть предложение ORDER BY, то оно обрабатывается.
Секционирование результирующего множества (Result Set Partitions). Аналитические функции позволяют пользователю разделить результирующее множество запроса на группы строк, называемых секциями. Термин "секция" (partitions) используется в другом смысле, отличающемся по значению от того, что используется при секционировании таблицы. Он применяется для обозначения групп, которые создаются после группировки предложением GROUP BY. Эти секции строятся в соответствии со значением колонки группировки.
Окна (Window). Для каждой строки в секции вы можете определить скользящее окно данных (sliding window of data). Это окно определяет интервал строк, используемых для вычислений от текущей строки. Размер окна может быть задан

либо как физическое число строк, либо как логическое число строк (условием). Окно имеет начальную строку (starting row) и конечную строку (ending row). В зависимости от определения окно может перемещаться по результирующему множеству в один или оба конца. Например, окно, определенное для функции кумулятивных сумм, будет иметь своей стартовой строкой первую строку секции, и ее конечная строка будет скользить от начальной точки через все к последней строке секции. В противоположность этому, окно, определенное для скользящего среднего, будет иметь и начальную, и конечную точки, скользящие так, чтобы захватывать постоянный физический или логический интервал.

Текущая строка (Current Row). Каждое вычисление с аналитическими функциями основывается на текущей строке в секции. Текущая строка поддерживается как ссылка на строку (позицию) внутри окна. Например, вычисление центрированного скользящего среднего может потребовать определения окна с 5-ю строками, предшествующими текущей, и 6-ю следующими за ней. Окно будет иметь размер в 12 строк.

Предложение OVER определяет секционирование и упорядочение набора строк до применения соответствующей аналитической функции. В качестве таких функций используются агрегатные и статистические, ранжирующие функции. Каждая из ранжирующих функций задействует предложение OVER.

Синтаксис:

Для агрегатных функций

< OVER_CLAUSE > ::= OVER ([PARTITION BY Value_expression, … [n]]

Для ранжирующих функций

< OVER_CLAUSE > ::= OVER ([PARTITION BY Value_expression, … [n]] <ORDER BY expression>)

PARTITION BY разделяет результирующий набор на секции. Аналитическая функция применяется к каждой секции отдельно, и вычисление начинается заново для каждой секции. Если это предложение опущено, функция интерпретирует все результирующее множество как одну группу.

Value_expression указывает столбец, по которому секционируется набор строк, произведенный соответствующим предложением FROM. Аргумент *Value_expression* может ссылаться только на столбцы, доступные через предложение FROM. Аргумент *Value_expression* не может ссылаться на выражения или псевдонимы в списке выбора. Выражение *Value_expression* может быть выражением столбца, скалярным вложенным запросом, скалярной функцией или пользовательской переменной.

Предложение *ORDER BY* задает порядок для ранжирующей функции. Если предложение *ORDER BY* используется в контексте ранжирующей функции, оно может ссылаться только на столбцы, доступные через предложение FROM. Указывать положение имени или псевдонима столбца в списке выборки с помощью целого числа нельзя. Предложение *ORDER BY* не может работать со статистическими аналитическими функциями.

В одном запросе с одним предложением FROM может использоваться несколько статистических или ранжирующих функций. Однако предложение OVER для каждой функции может применять свое секционирование и упорядочение.

Семантика NULL-значений соответствует семантике NULL-значений агрегатных функций.

3.2. Временная привязка данных и создание представления для таблицы фактов

В хранилищах данных обычно осуществляется временная привязка данных – загружаемая в хранилище информация соответствует конкретным моментам или интервалам времени. Компонент данных может быть привязан к дате продаж, а также к ежедневным, еженедельным, месячным и годовым итогам продаж. С этой целью в таблице фактов обычно присутствует столбец с типом данных дата-время (см. столбец ORDER_DATE в таблице OE.ORDERS). Однако для удобства последующей аналитической обработки полезно предварительно привязать данные в таблице фактов к элементарным временным интервалам, по которым можно затем обобщать и анализировать данные. К таким элементарным временным интервалам относятся *год, месяц, день, час, минута, секунда*. Начиная с Oracle 9i для извлечения значений года, месяца, дня, часа, минуты, секунды из данных типа дата-время применяется функция EXTRACT со следующим упрощенным синтаксисом:

EXTRACT (YEAR | MONTH | DAY | HOUR | MINUTE | SECOND FROM date_value),

где date_value – значение типа дата-время.
Пример 1: запрос на выборку из связанных таблиц OE.ORDERS и OE.ORDER_ITEMS (левое внешнее объединение) с выделением значений года, месяца, дня.

```
SELECT EXTRACT(YEAR FROM ord.ORDER_DATE) Year,
        EXTRACT(MONTH FROM ord.ORDER_DATE) Month,
        EXTRACT(DAY FROM ord.ORDER_DATE) Day,
        ord.ORDER_ID, ord.ORDER_DATE, ord.ORDER_MODE, ord.CUSTOMER_ID,
        ord.ORDER_STATUS, ord.ORDER_TOTAL, ord.SALES_REP_ID,
        oi.LINE_ITEM_ID, oi.PRODUCT_ID, oi.UNIT_PRICE, oi.QUANTITY
FROM OE.ORDERS ord,
        OE.ORDER_ITEMS oi
WHERE ord.ORDER_ID = oi.ORDER_ID (+);
```

Для удобства дальнейшей работы на основе этого запроса следует создать представление с именем **V_YMD_ORDERS** (см. описание лабораторной работы № 4.

3.3. Агрегатные и статистические функции

Агрегатные и статистические функции (обобщающие функции, групповые операции) обрабатывают по нескольку строк и возвращают один обобщенный результат. Фактически они позволяют объединять в той или иной форме сходные элементы информации, возвращаемые из базы данных. Для выполнения групповых операций Oracle предоставляет обобщающие функции и инструкции группировки (GROUP BY, HAVING и другие).
Синтаксис большинства обобщающих функций:

```
обобщающая_функция([DISTINCT | ALL] выражение)
```

Обобщающая_функция – указывает имя обобщающей функции:

- COUNT (количество),

- SUM (сумма),
- MAX (максимальное значение),
- MIN (минимальное значение),
- AVG (среднее значение),
- STDDEV (стандартное среднеквадратичное отклонение),
- VARIANCE (статистическая дисперсия).

DISTINCT – указывает, что обобщающая функция должна учитывать только неповторяющиеся значения *выражения*.
ALL – указывает, что обобщающая функция должна учитывать все значения *выражения*, в том числе и дублирующиеся. По умолчанию считается, что использовано ALL.
Выражение – указывает столбец или любое другое выражение, по которому необходимо выполнить обобщение.
Значения NULL для выражения обобщающими функциями игнорируются, в результат не входят.
Пример 2 – вычисляет количество записей в таблице ORDERS:

```
Select count(*) from OE.ORDERS;
```

Пример 3 – найти минимальное и максимальное значения по столбцу UNIT_PRICE во всем представлении V_YMD_ORDERS:

```
Select MIN(UNIT_PRICE), MAX(UNIT_PRICE) from OE.V_YMD_ORDERS;
```

Инструкция GROUP BY
Используется совместно с обобщающими функциями, разбивает результирующее множество на несколько групп, а затем для каждой группы выдается одна строка сводной информации. Если в списке SELECT присутствует смесь обобщенных и необобщенных значений, SQL считает, что нужно выполнить операцию GROUP BY, поэтому все необобщенные выражения должны быть указаны и в инструкции GROUP BY. Если этого не сделать, Oracle выдаст сообщение об ошибке.
Не разрешено использование групповой (обобщающей) функции в инструкции GROUP BY.
При группировке по столбцу, содержащему в некоторых строках NULL-значения, все строки с NULL-значениями помещаются в одну группу и представляются в выводе одной сводной строкой.
Для получения итоговых результатов с использованием инструкции GROUP BY существует возможность фильтрации записей таблицы при помощи инструкции WHERE. При выполнении оператора SQL, содержащего инструкции WHERE и GROUP BY, Oracle сначала применяет инструкцию WHERE и отсеивает строки, не удовлетворяющие условию WHERE. Затем строки, удовлетворяющие условию WHERE, группируются в соответствии с инструкцией GROUP BY. Синтаксис SQL требует, чтобы инструкция WHERE предшествовала инструкции GROUP BY.
Инструкция HAVING
Используется для наложения фильтра на группы, созданные инструкцией GROUP BY. Если запрос содержит GROUP BY и HAVING, результирующее множество будет содержать только те группы, которые удовлетворяют условию, указанному в инструкции HAVING. Синтаксис инструкции HAVING подобен синтаксису инструкции WHERE. Но для инструкции HAVING существует одно ограничение. Это условие (указанное в инструкции HAVING), может относиться только к выражениям списка SELECT или инструкции GROUP BY. Если в HAVING будет содержаться что-то, чего нет в SELECT или GROUP BY, будет выдано сообщение об ошибке.

Порядок следования инструкций GROUP BY и HAVING в операторе SELECT не имеет значения.
Можно использовать в одном запросе инструкции WHERE и HAVING. При этом важно понимать, как одна инструкция воздействует на другую. Инструкция WHERE выполняется первой, и строки, которые удовлетворяют условию WHERE, передаются в инструкцию GROUP BY. Инструкция GROUP BY сводит отфильтрованные данные в группы, а затем уже к группам применяется инструкция HAVING для устранения групп, не удовлетворяющих условию HAVING.
Синтаксис для запросов с обобщающими функциями и группировкой

```
SELECT столбец_1, столбец_n, обобщающая_функция(столбец)
FROM таблица
WHERE условие отбора строк
GROUP BY столбец_1, столбец_n
HAVING условие отбора групп
ORDER BY столбец
```

Пример 4. Вычислить количество заказов и общую их стоимость для клиентов по годам:

```
SELECT V.YEAR, V.CUSTOMER_ID, count(V.ORDER_ID), sum(V.ORDER_TOTAL)
FROM V_YMD_ORDERS V
GROUP BY V.YEAR, V.CUSTOMER_ID
HAVING V.YEAR BETWEEN 1996 AND 1999
ORDER BY V.YEAR, V.CUSTOMER_ID;
```

3.4. Аналитические функции перекрестного анализа таблиц

Функции ROLLUP и CUBE расширяют функциональность конструкции GROUP BY оператора SELECT.
Функция ROLLUP вычисляет промежуточные итоговые значения на любом уровне агрегирования, начиная с наиболее детализированного и кончая самым верхним.
Результат применения функции ROLLUP является достаточно очевидным. Эта функция вычисляет промежуточные итоговые значения, которые накапливаются (roll up), начиная с самых детализированных уровней группировки данных и заканчивая основным уровнем. Такая группировка осуществляется в соответствии со списком, который указывается в конструкции ROLLUP. Аргументом функции ROLLUP является упорядоченный список столбцов группировки. Вначале вычисляются стандартные групповые величины, указываемые в конструкции GROUP BY. Затем рассчитываются промежуточные итоговые значения для все более высоких уровней группировки. При этом последовательность уровней определяется списком столбцов группировки в направлении справа налево. После этого вычисляется общий итог.
Функция ROLLUP рассчитывает промежуточные итоговые значения для n + 1 уровней, где n — количество столбцов группировки.
Пример 5. Если в аргументе функции ROLLUP задаются такие столбцы группировки, как YEAR и CUSTOMER_ID (n = 2), результирующий набор данных будет содержать строки для трех уровней агрегирования:

```
SELECT V.YEAR, V.CUSTOMER_ID, count(V.ORDER_ID), sum(V.ORDER_TOTAL)
FROM V_YMD_ORDERS V
GROUP BY ROLLUP(V.YEAR, V.CUSTOMER_ID);
```

Функция CUBE представляющая собой расширение, аналогичное ROLLUP, позволяет использовать один оператор для вычисления всех возможных комбинаций промежуточных итогов. Простейший способ получить в Oracle полный набор промежуточных итогов, необходимых для перекрестного анализа таблиц, заключается в использовании функции CUBE. Эта функция позволяет использовать оператор SELECT, чтобы рассчитать промежуточные итоги для всех комбинаций группы измерений. С помощью данной функции рассчитываются также общие итоги. В результате получается набор данных, который обычно необходим для генерации любых кросстабличных отчетов. Таким образом, функция CUBE позволяет создать многотабличный отчет, использующий перекрестные ссылки, с помощью одного лишь оператора SELECT. Так же как и ROLLUP, функция CUBE является простым расширением конструкции GROUP BY.
Функция CUBE принимает в качестве аргумента набор столбцов группировки и рассчитывает промежуточные итоги для всех возможных комбинаций этих столбцов. Используя понятия многомерного анализа, можно сказать, что функция CUBE генерирует все промежуточные итоговые значения, которые могут быть рассчитаны для куба данных с заданными измерениями.
Так, функция CUBE с аргументами YEAR, CUSTOMER_ID (заказчик) и PRODUCT_ID (товар), возвратит результирующий набор данных, включающий все значения, которые были бы получены при использовании эквивалентного оператора ROLLUP, плюс дополнительные комбинации. Если в аргументе функции CUBE указаны n столбцов, то соответствующий запрос возвратит промежуточные итоговые значения, использующие 2n комбинаций.
Пример 6.

```
SELECT V.YEAR, V.CUSTOMER_ID, count(V.ORDER_ID), sum(V.ORDER_TOTAL)
FROM V_YMD_ORDERS V
GROUP BY CUBE(V.YEAR, V.CUSTOMER_ID);
```

Здесь используется тот же самый набор данных, что и в Примере 5 с функцией ROLLUP, однако набор результирующих комбинаций оказывается при этом намного более полным. Эти комбинации позволяют взглянуть на данные с самых разных точек зрения и служат основой для серьезного анализа информации. Такой анализ называется “slice and dice" (нарезка ломтиками и кубиками), поскольку он позволяет делать срезы данных в самых разных направлениях. Здесь есть все комбинации данных, которые могут присутствовать в перекрестной таблице. Эти результаты обеспечивают как детализацию, так и обобщение информации на различных уровнях.

3.5. Функции ранжирования

Функции ранжирования позволяют рассчитать категорию записи по отношению к другим записям в наборе данных. Они возвращают ранжирующее значение для каждой строки в секции. В зависимости от используемой функции значения некоторых строк могут совпадать. Есть четыре типа функций ранжирования:
Функция **RANK** – возвращает ранг каждой строки в секции результирующего набора. Ранг строки вычисляется как единица плюс количество рангов, находящихся до этой строки.
Функция **DENSE_RANK** – возвращает ранг строк в секции результирующего набора без промежутков в ранжировании. Ранг строки равен количеству различных значений рангов, которые предшествуют строке, увеличенному на единицу.

Функция **NTILE** – распределяет строки упорядоченной секции в заданное количество групп. Группы нумеруются, начиная с единицы. Для каждой строки функция NTILE возвращает номер группы, которой принадлежит строка.
Функция **ROW_NUMBER** – возвращает последовательный номер строки в секции результирующего набора, 1 соответствует первой строке в каждой из секций.
Функции RANK и DENSE_RANK позволяют ранжировать строки в группе. Существуют две функции ранжирования с синтаксисом:

RANK ( ) OVER ([< partition_by_clause >] < order_by_clause >)

DENSE_RANK ( ) OVER ([< partition_by_clause >] < order_by_clause >)

< partition_by_clause > ([PARTITION BY <value expression1> [, ...]]) делит результирующий набор, полученный с помощью предложения FROM, на секции, к которым применяется функция RANK.
< order_by_clause > (ORDER BY <value expression2> [collate clause] [ASC|DESC]) определяет порядок, в котором значения RANK применяются к строкам в секции. Целое значение не может представлять столбец < order_by_clause > при использовании его в ранжирующей функции.
Если две и более строки претендуют на один ранг, то все они получат одинаковый ранг. Функция RANK не всегда возвращает последовательные целые числа. Между номерами, возвращаемыми функцией DENSE_RANK, нет промежутков, и они всегда имеют последовательные значения ранга. Таким образом, различие между RANK и DENSE_RANK состоит в том, что DENSE_RANK не оставляет промежутков в ранжируемой последовательности.
Порядок сортировки, используемый для всего запроса, определяет порядок, в котором строки будут появляться в результирующем наборе.
Пример 7. Определить ранги заказчиков по количеству заказов с использованием функций RANK и DENSE_RANK:

```
SELECT V.CUSTOMER_ID, COUNT(V.ORDER_ID),
RANK () OVER(ORDER BY COUNT(V.ORDER_ID) DESC) AS Ranking,
DENSE_RANK () OVER
    (ORDER BY COUNT(V.ORDER_ID) DESC) AS Dense_Ranking
FROM V_YMD_ORDERS V
GROUP BY V.CUSTOMER_ID;
```

Пример 8. Определить ранги заказчиков по количеству заказов с использованием функций RANK и DENSE_RANK и с секционированием по годам:

```
SELECT V.YEAR, V.CUSTOMER_ID, COUNT(V.ORDER_ID),
RANK () OVER(PARTITION BY V.YEAR ORDER BY COUNT(V.ORDER_ID) DESC) AS
Ranking,
DENSE_RANK () OVER
    (PARTITION BY V.YEAR ORDER BY COUNT(V.ORDER_ID) DESC) AS
Dense_Ranking
FROM V_YMD_ORDERS V
WHERE V.YEAR BETWEEN 1998 AND 2000
GROUP BY V.YEAR, V.CUSTOMER_ID;
```

Функция ROW_NUMBER() назначает уникальный номер (последовательно, начиная с 1, в порядке, определенном ORDER BY) каждой строке в секции.

Синтаксис:

ROW_NUMBER ( ) OVER ([<partition_by_clause>] <order_by_clause>)

<partition_by_clause> делит результирующий набор, полученный по предложению FROM.
<order_by_clause> определяет порядок, в котором значение функции ROW_NUMBER назначается строкам в секции. Целое число не может представлять столбец, если аргумент <order_by_clause> используется в ранжирующей функции.
Предложение ORDER BY определяет последовательность, в которой строкам назначаются уникальные номера с помощью функции ROW_NUMBER в пределах указанной секции.
Пример 9. Определить ранги заказчиков по количеству заказов с использованием функций ROW_NUMBER, RANK и DENSE_RANK:

```
SELECT V.CUSTOMER_ID, COUNT(V.ORDER_ID),
ROW_NUMBER () OVER (ORDER BY COUNT(V.ORDER_ID) DESC) AS Row_Number_desc,
ROW_NUMBER () OVER (ORDER BY COUNT(V.ORDER_ID) ASC) AS Row_Number,
RANK () OVER(ORDER BY COUNT(V.ORDER_ID) DESC) AS Ranking,
DENSE_RANK () OVER
    (ORDER BY COUNT(V.ORDER_ID) DESC) AS Dense_Ranking
FROM V_YMD_ORDERS V
GROUP BY V.CUSTOMER_ID;
```

Как видно по результатам выполнения этого запроса, разница в поведении функций ранжирования проявляется на данных, где критерий определения места оказывается одинаковым у нескольких строк. ROW_NUMBER в таких случаях места раздает случайно.
Функция NTILE() делит упорядоченную секцию на указанное число групп, называемых **бакетами** (buckets), и назначает номер бакета каждой строке в секции. Для каждой строки функция NTILE() возвращает номер группы, которой принадлежит строка. NTILE() — очень полезная функция, поскольку дает возможность разделить набор данных на 40, 30 или другое число групп.
Бакеты вычисляются так, чтобы каждый из них имел одинаковое число строк, назначенное ему, или на одну больше. Например, если есть 100 строк в секции и находится функция NTILE для 4 бакетов, то 25 строк будет назначаться каждому бакету.
Если число строк в секции не делится нацело (на число бакетов), то число строк, назначаемое каждому бакету в начале, будет отличаться по крайней мере на 1. Например, если есть 103 строки в секции, к которой применяется функция NTILE(5), первые 21 строк будут назначены 1 бакету, следующие 21 — 2 бакету, следующие 21 — 3 бакету, следующие 20 — 4 бакету и последние 20 — 5 бакету.
Синтаксис:

NTILE (integer_expression) OVER ([<partition_by_clause>] < order_by_clause >)

integer_expression - положительное целое выражение-константа, указывающее количество групп, на которые необходимо разделить каждую секцию. Аргумент integer_expression может иметь тип int или bigint.
<partition_by_clause> делит результирующий набор, сформированный предложением FROM.

< order_by_clause > определяет порядок назначения значений функции NTILE строкам секции.
Аргумент integer_expression может ссылаться только на столбцы в предложении PARTITION BY и не может ссылаться на столбцы, перечисленные в текущем предложении FROM.
Пример 10. Пусть нам нужно провести группировку проданных товаров по объему продаж, причем количество заказанного товара не должно быть менее 100 единиц. Это можно сделать с помощью следующего запроса:

```
SELECT V.PRODUCT_ID, V.UNIT_PRICE * V.QUANTITY AS Amount,
NTILE (10) OVER(ORDER BY V.UNIT_PRICE * V.QUANTITY DESC) AS Tile_10
FROM V_YMD_ORDERS V
WHERE V.QUANTITY >= 100;
```

3.6. Функции подсчета долей

Функция CUME_DIST возвращает кумулятивное распределение значений в группе значений – относительную позицию значения в группе значений. Функция CUME_DIST вернет результат, который находится в интервале (0, 1] (0 в этот интервал не входит). Кумулятивная функция распределения (или просто функция распределения) в теории вероятностей однозначно задаёт распределение случайной величины или случайного вектора. Результат, возвращаемый функцией CUME_DIST, рассчитывается как отношение числа значений в наборе, не превышающих заданного значения, к размеру набора данных.
Синтаксис:

```
CUME_DIST ( ) OVER ( [ <partition_by_clause> ] < order_by_clause >)
```

<partition_by_clause> делит результирующий набор, полученный по предложению FROM.
<order_by_clause> определяет порядок, в котором значение функции CUME_DIST назначается строкам в секции.
Пример 11. Подсчитать относительную позицию суммы заказов для каждого заказчика из заданного множества.

```
SELECT V.CUSTOMER_ID, SUM(V.ORDER_TOTAL),
        CUME_DIST () OVER(ORDER BY SUM(V.ORDER_TOTAL) ASC) AS Cume_Dist
FROM V_YMD_ORDERS V
WHERE V.CUSTOMER_ID < 115
GROUP BY V.CUSTOMER_ID
ORDER BY SUM(V.ORDER_TOTAL) DESC;
```

В результате наибольшее значение попадает в наивысшую категорию 1, а к концу набора данных значение категории уменьшается до минимальной величины 0,11.
Функция PERCENT_RANK так же вычисляет относительную позицию указанных величин в наборе данных. Алгоритм, положенный в основу функции PERCENT_RANK, несколько отличается и основывается на подсчете числа строк. Функция PERCENT_RANK вернет результат, который находится в интервале [0, 1] (0 в этот интервал входит).
Синтаксис:

```
PERCENT_RANK ( ) OVER ( [ <partition_by_clause> ] < order_by_clause >)
```

<partition_by_clause> делит результирующий набор, полученный по предложению FROM.
<order_by_clause> определяет порядок, в котором значение функции CUME_DIST назначается строкам в секции.
Пример 12. Подсчитать относительную позицию суммы заказов для каждого заказчика из заданного множества (аналогично примеру 11).

```
SELECT V.CUSTOMER_ID, SUM(V.ORDER_TOTAL),
        PERCENT_RANK () OVER(ORDER BY SUM(V.ORDER_TOTAL) ASC) AS
        Percent_Rank
FROM V_YMD_ORDERS V
WHERE V.CUSTOMER_ID < 115
GROUP BY V.CUSTOMER_ID
ORDER BY SUM(V.ORDER_TOTAL) DESC;
```

В результате наибольшее значение попадает в наивысшую категорию 1, а к концу набора данных значение категории уменьшается до минимальной величины 0.
Пример 13. Подсчитать относительную позицию суммы заказов для каждого заказчика из заданного множества отдельно для двух лет.

```
SELECT V.YEAR, V.CUSTOMER_ID, SUM(V.ORDER_TOTAL),
        PERCENT_RANK () OVER(PARTITION BY V.YEAR ORDER BY
        SUM(V.ORDER_TOTAL) ASC) AS Percent_Rank
FROM V_YMD_ORDERS V
WHERE V.CUSTOMER_ID < 115
   AND V.YEAR BETWEEN 1999 AND 2000
GROUP BY V.YEAR, V.CUSTOMER_ID
ORDER BY V.YEAR, SUM(V.ORDER_TOTAL) DESC;
```

В результате для каждого года наибольшее значение попадает в наивысшую категорию 1, а к концу набора данных значение категории уменьшается до минимальной величины 0.
Пример 14. Подсчитать относительную позицию суммы заказов для двух заказчиков отдельно для двух лет.

```
SELECT V.YEAR, V.CUSTOMER_ID, SUM(V.ORDER_TOTAL),
        PERCENT_RANK () OVER(PARTITION BY V.YEAR ORDER BY
        SUM(V.ORDER_TOTAL) ASC) AS Percent_Rank
FROM V_YMD_ORDERS V
WHERE V.CUSTOMER_ID < 115
   AND V.YEAR BETWEEN 1999 AND 2000
GROUP BY V.YEAR, V.CUSTOMER_ID
HAVING V.CUSTOMER_ID IN (104, 105, 109)
ORDER BY V.YEAR, SUM(V.ORDER_TOTAL) DESC;
```

Функции обработки данных методом окна позволяют получить аналитические данные об изменении параметров исследуемого объекта (бизнеса) с течением времени. Обычно выполняется анализ, основанный на расчете средних и итоговых значений в зависимости от времени. Эти функции должны употребляться в конструкциях SELECT и ORDER BY, входящие в запросы. Для некоторых аналитических функций, например, агрегирующих, можно дополнительно указать

объем строк, участвующих в вычислении, выполняемом для каждой строки в группе. Этот объем, своего рода контекст строки, называется "окном", а границы окна могут задаваться различными способами.

4. Меры безопасности

Во время выполнения лабораторной работы необходимо:

- соблюдать правила включения и выключения вычислительной техники;
- не подключать кабели, разъемы и другую аппаратуру к компьютеру, не относящиеся к лабораторной установке;
- при включенном напряжении сети не отключать, не подключать и не трогать кабели, соединяющие различные устройства компьютера;
- в случае обнаруженной неисправности в работе оборудования или нарушения правил техники безопасности сообщить руководителю лабораторной работы;
- не пытаться самостоятельно устранить неисправности в работе аппаратуры;
- по окончании работы привести в порядок рабочее место.

ВНИМАНИЕ! При работе за компьютером необходимо помнить: к каждому рабочему месту подведено опасное для жизни напряжение. Поэтому во время работы надо быть предельно внимательным и соблюдать все требования техники безопасности!

5. Задание

1. С помощью инструмента SQL Developer подключиться к базе данных под именем OE. Выполнить Пример 1 и на основе этого запроса создать представление с именем V_YMD_ORDERS.
2. Последовательно выполнить все примеры с 2 по 14, проанализировать полученные результаты.
3. Удалить представление V_YMD_ORDERS.

6. Требования к содержанию и оформлению отчета

Отчет должен быть выполнен в текстовом редакторе MS Word. Отчет должен содержать:

- Краткие теоретические сведения.
- Тексты всех SQL-команд с комментариями относительно каждой использованной в запросе инструкции.
- Скриншоты выполнения всех SQL-команд.
- Результирующие таблицы с данными для всех выполненных запросов и команд.
- Выводы по проделанной работе.

7. Контрольные вопросы

1. Для чего применяются аналитические функции?
2. Какие существуют виды аналитических функций Oracle?
3. Каковы правила использования агрегатных и статистических функций?

4. Как работают функции перекрестного анализа данных?
5. Как работают ранжирующие функции?
6. Как работают функции подсчета долей?